Tema 5

Tratamiento de datos

Base de Datos

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Inserción, borrado y modificación de registros 5
2.1 Inserción de datos 5
2.1.1 Inserción a partir de datos proporcionados por el usuario 6
2.1.2 Inserción a partir de datos recuperados de subconsultas 8
2.2 Modificación de datos 12
2.2.1 Modificación de datos a partir de datos proporcionados por el usuario 13
2.2.2 Modificación de datos a partir de datos recuperados de subconsultas 14
2.3 Borrado de datos 17
2.3.1 Borrado de datos a partir de datos proporcionados por el usuario 18
2.3.2 Borrado de datos a partir de datos recuperados de subconsultas 19
3. Borrados y modificaciones e integridad referencial 25
4. Subconsultas y composiciones en órdenes de edición 32
4.1 Subconsultas de edición 32
4.2 Composiciones 34
5. Transacciones. Sentencias de confirmación y deshacer 37
6. Políticas de bloqueo: a nivel de registro, a nivel de tabla 40
7. Resumen 48
Glosario 49
Bibliografía 50

Objetivos de la unidad

El objetivo de este tema es proporcionar al lector una comprensión profunda y práctica de cómo trabajar con bases de datos MySQL, desde la inserción, borrado y modificación de registros hasta la gestión de transacciones y políticas de bloqueo.

Los principales objetivos de la unidad incluyen:

- **Inserción, borrado y modificación de registros:** Aprender cómo agregar, actualizar y eliminar registros en una base de datos MySQL, utilizando diferentes tipos de consultas y operaciones para manipular los datos de manera eficiente.
- **Gestión de transacciones:** Comprender cómo las transacciones funcionan en MySQL y cómo utilizarlas para asegurar la integridad de los datos y mantener la coherencia de las operaciones.
- **Integridad referencial en modificación y borrado de datos:** Aprender cómo gestionar políticas de bloqueo para evitar conflictos en una base de datos MySQL y asegurar que los usuarios accedan a los datos de manera segura y eficiente.

Tech Hub <FP> euroformac

1. Introducción

MySQL es un sistema de gestión de bases de datos relacional de código abierto muy popular y ampliamente utilizado en aplicaciones web y de servidor. Una de las características clave de MySQL es su capacidad para gestionar usuarios y sus permisos para acceder a la base de datos. En este manual, se proporcionará una guía detallada sobre la gestión de usuarios en MySQL, desde la creación y eliminación de usuarios hasta la asignación de permisos y la gestión de contraseñas.

2. Inserción, borrado y modificación de registros

2.1 Inserción de datos

La sentencia **INSERT** en MySQL se utiliza para insertar uno o varios registros en una tabla. La sintaxis básica de la sentencia INSERT es la siguiente:

INSERT INTO nombre_tabla (columna1, columna2, ..., columnaN) VALUES (valor1, valor2, ..., valorN);

Dónde:

- **nombre_tabla** es el nombre de la tabla donde se desea insertar el registro
- **columna1, columna2, ..., columnaN** son los nombres que se insertarán en las respectivas columnas.
- **valor1, valor 2, ..., valorN** son los valores que se insertarán en las respectivas columnas.

Características de la sentencia INSERT en MySQL:

- Se puede insertar uno o varios registros a la vez con `INSERT INTO tabla (…) VALUES (…), (…), (…)`.
- Es necesario especificar el nombre de la tabla y las columnas donde se insertarán los valores.
- Los valores a insertar pueden ser literales o variables.
- Si no se especifica un valor para una columna, se insertará el valor predeterminado de la columna (si está definido).
- Si se intenta insertar un valor que no cumple con las restricciones de la columna (por ejemplo, un valor que excede el límite de caracteres), MySQL generará un error y la inserción fallará.

- Si la tabla tiene claves primarias u otras restricciones de integridad referencial, se deben cumplir para que se pueda insertar el registro.

- Si se intenta insertar un registro con un valor duplicado en una columna que tiene una restricción única, MySQL generará un error y la inserción fallará.

Limitaciones de la sentencia INSERT en MySQL:

- No se puede insertar datos directamente desde otra tabla en una sola sentencia INSERT. En su lugar, se puede utilizar una sentencia INSERT INTO SELECT.
- Si se desea insertar varios registros a la vez, es necesario especificar los valores para todas las columnas en cada registro. No se pueden omitir columnas en algunos registros y especificarlas en otros.

2.1.1 Inserción a partir de datos proporcionados por el usuario

Consiste en agregar datos a partir de formularios o cualquier forma de entrada, que los usuarios introducen para que queden persistido en una o varias tablas de la base de datos.

A continuación, veamos algunos ejemplos:

Ejemplo 1.

```
INSERT INTO Usuarios (Id, Nombre, Apellido1, Apellido2, fechaNacimiento) VALUES (1, 'Juan', 'García', 'Pérez', '1990-05-20');
```

En este ejemplo, estamos insertando un registro en la tabla **Usuarios:**

- El valor literal 1 se inserta en el campo Id
- El valor literal 'Juan' se inserta en el campo Nombre
- El valor literal 'García' se inserta en el campo Apellido1
- El valor literal 'Pérez' se inserta en el campo Apellido2
- El valor literal '1990-05-20' se inserta en el campo fechaNacimiento.

Importante

Ten en cuenta que el formato de fecha debe ser AAAA-MM-DD para ser compatible con MySQL. Si se intenta insertar una fecha en un formato diferente, MySQL generará un error.

Ejemplo 2.

En este ejemplo veremos cómo insertar datos en la tabla de **Productos** de una base de datos MySQL, considerando que la tabla está formada por las columnas **Id, Nombre, ReferenciaInterna, IdCliente y FechaCaducidad**:

```
INSERT INTO Productos (Id, Nombre, ReferenciaInterna, IdCliente, FechaCaducidad)
VALUES (1, 'Producto A', 'REF001', 123, '2023-12-31');
```

En este ejemplo se está insertando un registro en la tabla de Productos con los siguientes valores:

- Id: 1
- Nombre: 'Producto A'
- ReferenciaInterna: 'REF001'
- IdCliente: 123
- FechaCaducidad: '2023-12-31'

Importante

Es importante tener en cuenta que, al insertar valores en una tabla, es necesario asegurarse de que los tipos de datos de valores a insertar sean compatibles con los tipos de datos definidos para cada columna.

Ejemplo 3.

En este tercer ejemplo insertaremos datos en la tabla de **Aulas** de una base de datos MySQL, considerando que la tabla está formada por las columnas **Id, NombreAula, IdEdificio, NumeroPlanta y CapacidadMaxima**:

```
INSERT INTO Aulas (Id, NombreAula, IdEdificio, NumeroPlanta, CapacidadMaxima)
VALUES (1, 'Aula 101', 1, 1, 30);
```

En este ejemplo se está insertando un registro en la tabla de Aulas con los siguientes valores:

- Id: 1
- NombreAula: 'Aula 101'

- IdEdificio: 1
- NumeroPlanta: 1
- CapacidadMaxima: 30

Importante

Es muy recomendable que las tablas de bases de datos relacionales dispongan siempre de una clave primaria. También es una práctica habitual que las claves primarias sean auto numeradas, y que el usuario no debe preocuparse de asignarle un valor único cada vez que necesita insertar un nuevo registro.

Presuponiendo que el campo **Id** es la clave primaria de los ejemplos anteriores, podríamos reescribir los ejemplos anteriores de esta manera:

INSERT INTO Usuarios (Id, Nombre, Apellido1, Apellido2, fechaNacimiento) VALUES (**NULL**, 'Juan', 'García', 'Pérez', '1990-05-20');

INSERT INTO Productos (Id, Nombre, ReferenciaInterna, IdCliente, FechaCaducidad)

VALUES (**NULL**, 'Producto A', 'REF001', 123, '2023-12-31');

INSERT INTO Aulas (Id, NombreAula, IdEdificio, NumeroPlanta, CapacidadMaxima)

VALUES (**NULL**, 'Aula 101', 1, 1, 30);

Especificando como valor la palabra reservada NULL, en campos de clave primaria auto numerados, le estamos indicando a MySQL que le asigne un valor único de manera automática.

2.1.2 Inserción a partir de datos recuperados de subconsultas

MySQL permite la inserción de datos a partir de **subconsultas**, que son consultas internas que se realizan dentro de la consulta principal.

Esto permite recuperar datos de otras tablas o consultas, y utilizarlos para insertar nuevos registros en la tabla de destino.

La **sintaxis** general para una **consulta de inserción con subconsulta** es la siguiente:

```
INSERT INTO tabla_destino (columna1, columna2, columna3, ...)
SELECT subconsulta.columna1, subconsulta.columna2, subconsulta.columna3, ...
FROM tabla_origen AS subconsulta
WHERE condición;
```

En esta consulta: la subconsulta se realiza primero y los resultados se utilizan para insertar los datos en la tabla de destino especificada.

La cláusula WHERE se utiliza para filtrar los resultados de la subconsulta, solamente si es necesario.

Entre las **características** de la **inserción a partir de subconsultas**, podemos mencionar las siguientes:

- Es una forma eficiente de insertar grandes cantidades de datos en una tabla, ya que permite recuperarlos y transformarlos antes de insertarlos.
- Permite realizar inserciones más complejas, que requieren la recuperación de datos de varias tablas o consultas.
- Puede ser útil para realizar copias de seguridad de datos, o para transferir datos entre diferentes bases de datos.

Sin embargo, también existen algunas **limitaciones** a tener en cuenta:

- Las subconsultas pueden ser más lentas que las consultas directas, ya que implican una mayor carga de procesamiento.
- Es necesario tener en cuenta la compatibilidad de los tipos de datos entre la subconsulta y la tabla de destino, para evitar errores de inserción.
- Es importante asegurarse de que las subconsultas devuelvan un número adecuado de columnas, y que los nombres y órdenes de las columnas coincidan con las de la tabla de destino.

Importante

Aunque la inserción a partir de subconsultas es una herramienta útil y poderosa para manipular datos en MySQL, debe ser utilizada con precaución para realizar operaciones complejas de manera eficiente.

A continuación, veamos algunos ejemplos:

Ejemplo 1.

Inserción en la tabla de Usuarios a partir de una subconsulta en MySQL:

Supongamos que tenemos una tabla llamada Clientes, que contiene información sobre los clientes de una tienda y queremos insertar un nuevo registro en la tabla de Usuarios con la información del primer cliente de la tabla Clientes. La consulta sería la siguiente:

```
INSERT INTO Usuarios (Id, Nombre, Apellido1, Apellido2, fechaNacimiento)
SELECT 1, Nombre, Apellido1, Apellido2, fechaNacimiento
FROM Clientes
ORDER BY Id ASC
LIMIT 1;
```

En esta consulta, la subconsulta se realiza en la tabla Clientes, donde se selecciona el primer registro **ordenando por Id** de **forma ascendente y limitando** el resultado a un único registro.

A continuación, se inserta el registro recuperado en la tabla de Usuarios, con el Id y los valores de las columnas de los demás campos.

Importante

Es importante tener en cuenta que, en la consulta de inserción con subconsulta, las columnas de la subconsulta deben coincidir con las columnas de la tabla de destino. En el ejemplo 1, asumimos que la tabla Clientes tiene las mismas columnas que la tabla Usuarios.

Ejemplo 2.

Ejemplo de cómo realizar una inserción en la tabla de Productos a partir de una subconsulta que consulta la tabla de ProductosSinCatalogar en MySQL:

Supongamos que tenemos dos tablas: ProductosSinCatalogar y Productos.

La tabla de ProductosSinCatalogar contiene información sobre los productos que aún no han sido catalogados y la tabla de Productos está vacía.

Queremos insertar todos los registros de la tabla de productos sin catalogar en la tabla de Productos. La consulta sería la siguiente:

```
INSERT INTO Productos (Id, Nombre, ReferenciaInterna, IdCliente, FechaCaducidad)
SELECT psc.IdProducto, psc.NombreProducto, psc.ReferenciaInterna, NULL, psc.FechaCaducidad
FROM ProductosSinCatalogar AS psc
WHERE NOT EXISTS (
   SELECT *
   FROM Productos AS p
   WHERE psc.IdProducto = p.Id
) ;
```

En esta consulta, la subconsulta se realiza en la tabla ProductosSinCatalogar, donde se seleccionan todos los productos no catalogados que aún no ha sido insertado en la tabla de Productos.

2.2 Modificación de datos

La modificación de datos en MySQL se refiere a la capacidad de cambiar o actualizar registros existentes en una tabla. Esto se puede hacer utilizando la instrucción **UPDATE**.

La **sintaxis básica** para modificar un registro es la siguiente:

```
UPDATE nombre_tabla
SET columna1 = valor1, columna2 = valor2, ...
WHERE condición;
```

Donde:

- **nombre_tabla** es el nombre de la tabla que se quiere modificar
- **columna1, columna2, etc.** son los nombres de las columnas que se quieren modificar
- **valor1, valor2, etc.** son los nuevos valores que se quieren asignar a las columnas
- **condición** es una expresión que determina qué registros se deben modificar.

Características de la sentencia UPDATE en MySQL:

- La posibilidad de modificar una o varias columnas de un registro.
- La capacidad de utilizar expresiones y funciones para calcular los nuevos valores de las columnas.
- La posibilidad de modificar varios registros a la vez utilizando una única instrucción UPDATE (con una condición que incluya muchos registros o, simplemente, sin poner ninguna condición, lo que implicará modificar todos los registros de una tabla).

Limitaciones de la sentencia UPDATE en MySQL:

- La modificación de registros puede ser un proceso intensivo en términos de recursos si se están actualizando grandes cantidades de datos.
- La modificación de registros puede afectar la integridad de los datos si no se tiene cuidado al escribir la cláusula WHERE.
- La modificación de registros puede estar limitada por permisos y restricciones de seguridad en la base de datos.

2.2.1 Modificación de datos a partir de datos proporcionados por el usuario

Consiste en modificar datos a partir de formularios, o cualquier forma de entrada, que los usuarios introducen para que queden persistidos en una tabla de la base de datos.

Veamos algunos ejemplos:

Ejemplo 1.

Ejemplo de cómo realizar una actualización en la tabla de Usuarios en MySQL

Supongamos que queremos actualizar el apellido1 y el apellido2 del usuario con Id igual a 1.

La consulta sería la siguiente:

```
UPDATE Usuarios
SET Apellido1 = 'Pérez', Apellido2 = 'González'
WHERE Id = 1;
```

En este ejemplo, estamos actualizando los valores de las columnas Apellido1 y Apellido2 en la tabla Usuarios donde el Id sea igual a 1.

Si queremos actualizar más de una columna al mismo tiempo, solo es necesario agregarlas separadas por comas en la cláusula SET.

Por ejemplo, si queremos actualizar también el nombre del usuario con Id igual a 1, la consulta sería:

```
UPDATE Usuarios
SET Nombre = 'Juan', Apellido1 = 'Pérez', Apellido2 = 'González'
WHERE Id = 1;
```

En este caso, estamos actualizando los valores de las columnas Nombre, Apellido1 y Apellido2 en la tabla Usuarios donde el Id sea igual a 1.

Ejemplo 2.

En este ejemplo veremos cómo modificar datos en la tabla de **Productos** de una base de datos MySQL, considerando que la tabla está formada por las columnas **Id, Nombre, ReferenciaInterna, IdCliente y FechaCaducidad.**

Supongamos que queremos modificar el nombre y la fecha de caducidad de un producto cuyo ID es 12345. El siguiente código podría usarse para realizar la modificación:

```
UPDATE Productos

SET Nombre = 'Nuevo Nombre', FechaCaducidad = '2024-01-01'

WHERE Id = 12345;
```

En este código:

- "Productos" es el nombre de la tabla
- "Nombre" y "FechaCaducidad" son los nombres de las columnas que queremos modificar
- "Id" es el nombre de la columna que identifica al producto que queremos modificar.
- El valor "12345" es el valor del ID del producto que queremos modificar.

Al ejecutar esta sentencia SQL la base de datos actualizará los valores de la columna "Nombre" y "FechaCaducidad" del producto con el ID 12345, asignándoles los nuevos valores de "Nuevo Nombre" y "2024-01-01", respectivamente.

2.2.2 Modificación de datos a partir de datos recuperados de subconsultas

La **modificación a partir de datos recuperados de subconsultas** es una técnica que se utiliza en SQL para actualizar los valores de una tabla en función de los resultados obtenidos por una subconsulta.

Una subconsulta es una consulta que se realiza dentro de otra consulta, y que devuelve un conjunto de resultados que se pueden utilizar en la consulta principal.

En el contexto de la modificación de datos, una subconsulta se utiliza para recuperar los valores que se van a utilizar para actualizar las filas de una tabla.

Importante

En MySQL la sintaxis para la modificación de datos a partir de subconsultas se realiza utilizando la cláusula **UPDATE**, seguida del nombre de la tabla que se va a modificar, y la cláusula **SET**, que especifica los valores que se van a actualizar.

La subconsulta se utiliza dentro de una expresión WHERE para seleccionar las filas que se van a actualizar.

La sintaxis general para la modificación a partir de subconsultas en MySQL es la siguiente:

```
UPDATE tabla1
SET columna1 = valor1, columna2 = valor2, ...
WHERE columna3 = (SELECT columna4 FROM tabla2 WHERE condicion);
```

En esta sintaxis:

- **tabla1** es el nombre de la table que se va a modificar.
- **columna1, columna2, etc.,** son los nombres de las columnas que se van a actualizar
- **valor1, valor2, etc.,** son los valores que se van a asignar a esas columnas.
- **columna3** es la columna de la tabla1 que se va a comparer con los resultados de las subconsultas.
- **columna4** es la table que se utiliza en la subconsulta.
- **condición** es la condición que se utiliza para filtrar los resultados de la subconsulta.

Características principales de la modificación de datos a partir de subconsultas:

- Permite actualizar las filas de una tabla utilizando los resultados de una subconsulta.
- Es una técnica flexible que permite realizar actualizaciones complejas y precisas en función de las necesidades específicas del usuario.
- Se puede utilizar en combinación con otras técnicas de SQL, como las funciones de agregación y los operadores lógicos, para obtener resultados más precisos.

Límites de la modificación de datos a partir de subconsultas:

- Puede ser difícil de leer y entender, especialmente en casos donde la subconsulta es muy compleja.
- Puede ser menos eficiente que otras técnicas de modificación de datos, especialmente cuando se utilizan subconsultas grandes o cuando se actualizan muchas filas de la tabla.
- En algunos casos, las bases de datos pueden no permitir la modificación a partir de subconsultas por razones de seguridad o rendimiento.

Ejemplo.

```
UPDATE productos
SET precio = precio * 1.1
WHERE categoria_id = (SELECT id FROM categorias WHERE nombre = 'Electrónica');
```

Esta consulta actualiza: los precios de los productos en un 10% para aquellos que pertenecen a la categoría "Electrónica" que se encuentra en la tabla "categorias".

La subconsulta se utiliza para recuperar el ID de la categoría "Electrónica" y compararlo con los valores de la columna "categoria_id" de la tabla "productos".

2.3 Borrado de datos

El **borrado de datos en MySQL** es una operación que elimina una o varias filas de una tabla en una base de datos. Esta operación se realiza utilizando la cláusula **DELETE**.

Características principales del borrado de datos en MySQL:

- La cláusula DELETE permite eliminar una o varias filas de una tabla en función de una o varias condiciones especificadas.
- Es una operación irreversible que elimina los datos de forma permanente de la base de datos.
- Puede ser utilizado en combinación con otras cláusulas de SQL, como WHERE, ORDER BY, y LIMIT, para realizar borrados más precisos.
- El borrado de datos se puede realizar en una sola tabla o en varias tablas a través de una consulta JOIN:

```
DELETE p
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
WHERE c.activo = 0;
```

Limitaciones del borrado de datos en MySQL:

- Es una operación potencialmente peligrosa que puede resultar en la pérdida de datos importantes si no se realiza correctamente.
- Si se eliminan muchos datos, esto puede afectar el rendimiento de la base de datos y el tiempo de respuesta de las consultas.
- En algunas bases de datos, la eliminación de datos puede requerir permisos especiales o privilegios de administrador.

2.3.1 Borrado de datos a partir de datos proporcionados por el usuario

Como dijimos anteriormente, si deseamos borrar datos a partir de datos proporcionados por el usuario en MySQL, podemos utilizar una sentencia DELETE que especifique una cláusula WHERE para identificar los registros que desea eliminar.

Ejemplo 1.

Supongamos que tiene una tabla **"usuarios"** con columnas **"id_usuario", "nombre" y "apellido"**.

Si deseamos eliminar el registro correspondiente a un usuario específico cuyo ID es proporcionado por el usuario, siendo este el 1245, podemos utilizar la siguiente sentencia **DELETE**:

```
DELETE FROM usuarios
WHERE id_usuario = 1245
```

Importante

La cláusula WHERE es importante para garantizar que solo se eliminen los registros específicos que deseamos eliminar y no todos los registros de la tabla.

También es importante tener en cuenta que una vez que se eliminan los datos no se pueden recuperar, por lo que siempre es recomendable hacer una copia de seguridad de los datos antes de eliminarlos.

Ejemplo 2.

Eliminar un registro específico de una tabla usando la cláusula **WHERE**.

Supongamos que tenemos una tabla llamada **"productos"** que contiene información sobre los productos de nuestra tienda.

Si deseamos eliminar un producto específico de la tabla, podemos usar la siguiente sentencia **DELETE**:

```
DELETE FROM productos
WHERE id_producto = 4602;
```

En este ejemplo, "4602" es el ID del producto que desea eliminar.

La sentencia **DELETE** elimina el registro que coincide con ese ID de la tabla "productos".

Ejemplo 3.

Eliminar múltiples registros de una tabla usando una cláusula **WHERE**.

Supongamos que tenemos una tabla llamada "ventas" que contiene información sobre todas las ventas realizadas en nuestra tienda.

Si deseamos eliminar todas las ventas realizadas antes de una fecha específica, podemos usar la siguiente sentencia **DELETE**:

```
DELETE FROM ventas
WHERE fecha_venta < '2023-03-31';
```

En este ejemplo, la cláusula **WHERE** selecciona todos los registros de la tabla "ventas" que tienen una fecha de venta anterior al 1 de marzo de 2023.

La sentencia **DELETE** elimina todos los registros que cumplen con ese criterio.

2.3.2 Borrado de datos a partir de datos recuperados de subconsultas

En MySQL es posible utilizar subconsultas para recuperar datos de una o varias tablas y usar estos datos en una sentencia **DELETE** para eliminar registros específicos.

Veamos algunos ejemplos:

Ejemplo 1.

Supongamos que tenemos dos tablas: "clientes" y "pedidos".

La tabla "clientes" contiene información sobre los clientes y la tabla "pedidos" contiene información sobre los pedidos realizados por esos clientes.

Si desea eliminar todos los pedidos realizados por un cliente específico, podemos usar una subconsulta en la siguiente sentencia **DELETE**:

```
DELETE FROM pedidos
WHERE id_cliente = (
  SELECT id_cliente
```

```
    FROM clientes
    WHERE nombre = Alfonso'
);
```

En este ejemplo, la subconsulta selecciona el ID del cliente con nombre "Alfonso" de la tabla "clientes".

La sentencia **DELETE** utiliza este ID para eliminar todos los registros de la tabla "pedidos" que corresponden a ese cliente.

Ejemplo 2.

Supongamos que tenemos dos tablas: "usuarios" y "posts".

La tabla "usuarios" contiene información sobre los usuarios y la tabla "posts" contiene información sobre los posts que han creado.

Si deseamos eliminar todos los posts de los usuarios que no tienen actividad desde el año 2020, podemos usar la siguiente sentencia **DELETE** con una subconsulta:

```
DELETE FROM posts
WHERE id_usuario IN (
    SELECT id_usuario
    FROM usuarios
    WHERE YEAR(ultima_actividad) <= 2020
);
```

La subconsulta selecciona el ID de todos los usuarios que no han tenido actividad desde el año 2020.

La sentencia **DELETE** utiliza estos IDs para eliminar todos los registros de la tabla "posts" que corresponden a los usuarios inactivos.

Importante

Es importante tener en cuenta que las subconsultas pueden tener un impacto significativo en el rendimiento de la base de datos, especialmente si la subconsulta devuelve un gran número de registros o si se usan varias subconsultas en una sola sentencia.

Por esta razón, es importante optimizar cuidadosamente las subconsultas para garantizar un rendimiento óptimo de la base de datos.

Veamos en el siguiente ejemplo práctico cómo se lleva a cabo la gestión de membresías en el gimnasio "FitLife"

FitLife es un gimnasio que mantiene dos tablas principales en MySQL:

- **Miembros:** datos personales de cada socio
- **Membresías:** información de la suscripción de cada miembro

Las estructuras simplificadas son:

```
CREATE TABLE Miembros (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  apellidos VARCHAR(100),
  fecha_nacimiento DATE,
  fecha_inscripcion DATE
);

CREATE TABLE Membresías (
  id INT AUTO_INCREMENT PRIMARY KEY,
  miembro_id INT,
  tipo VARCHAR(20),          -- 'Mensual', 'Trimestral', 'Anual'
  fecha_inicio DATE,
  fecha_fin DATE,
  FOREIGN KEY (miembro_id) REFERENCES Miembros(id)
);
```

A continuación, resolveremos las siguientes tareas de inserción, modificación y borrado:

Preguntas

1. Inserción
   a) Dar de alta a tres nuevos socios:
      - Laura Gómez (nacida el 1988-07-15), inscrita hoy (2025-07-13).
      - Carlos Ruiz (nacido el 1992-03-10), inscrito hoy.

- Elena Martín (nacida el 1975-11-22), inscrita hoy.

b) Asignarles a cada uno una membresía mensual que comienza hoy y finaliza dentro de un mes (2025-08-13).

2. Modificación

- El socio Carlos Ruiz decide cambiar su membresía de "Mensual" a "Trimestral". Actualizar su registro de membresía para que ahora expire el 2025-10-13.

3. Borrado

Dar de baja automáticamente todas las membresías cuyo fecha_fin sea anterior al 2025-06-01 (es decir, suscripciones caducadas hace más de un mes).

Tech Hub <FP> euroformac

Soluciones

1. **Inserción de datos**

a) **Insertar nuevos miembros**

INSERT INTO Miembros (nombre, apellidos, fecha_nacimiento, fecha_inscripcion)

VALUES

('Laura', 'Gómez', '1988-07-15', '2025-07-13'),

('Carlos', 'Ruiz', '1992-03-10', '2025-07-13'),

('Elena', 'Martín', '1975-11-22', '2025-07-13');

Aquí usamos la forma múltiple VALUES (…), (…), (…) para agregar tres filas de golpe.

b) **Asignar membresías mensuales**

INSERT INTO Membresías (miembro_id, tipo, fecha_inicio, fecha_fin)

VALUES

(LAST_INSERT_ID() - 2, 'Mensual', '2025-07-13', '2025-08-13'),

(LAST_INSERT_ID() - 1, 'Mensual', '2025-07-13', '2025-08-13'),

(LAST_INSERT_ID(), 'Mensual', '2025-07-13', '2025-08-13');

Usamos LAST_INSERT_ID() para referirnos a los últimos IDs generados; restamos 2 y 1 para los tres miembros insertados en bloque.

2. **Modificación de datos**

Actualizar la membresía de Carlos Ruiz a trimestral

UPDATE Membresías m

JOIN Miembros p ON m.miembro_id = p.id

SET

m.tipo = 'Trimestral',

m.fecha_fin = '2025-10-13'

WHERE

p.nombre = 'Carlos'

AND p.apellidos = 'Ruiz';

Aquí empleamos un UPDATE … JOIN para localizar la fila de membresía asociada a Carlos Ruiz.

3. Borrado de datos

Eliminar membresías expiradas antes del 2025-06-01

```
DELETE m
FROM Membresías m
WHERE m.fecha_fin < '2025-06-01';
```

Con este DELETE … FROM eliminamos únicamente las filas de la tabla Membresías cuya fecha_fin sea anterior al 1 de junio de 2025.

3. Borrados y modificaciones e integridad referencial

En las bases de datos relacionales las operaciones de borrado y modificación de datos pueden tener un impacto en la integridad referencial de la base de datos.

Definición

La **integridad referencial** en las bases de datos es una propiedad que garantiza que todas las relaciones entre las tablas de la base de datos sean coherentes y estén bien definidas.

Dicho de otro modo, la **integridad referencial** garantiza que los datos almacenados en una base de datos relacionales sean precisos y consistentes, y que se mantengan en su estado correcto incluso después de la realización de operaciones de inserción, actualización o borrado.

La integridad referencial se basa en la relación entre las tablas de una base de datos.

En una base de datos relacional una tabla puede tener una o varias claves primarias, que identifican de forma única cada registro en la tabla.

Además de estas claves primarias, una tabla puede tener una o varias claves foráneas, que establecen una relación entre la tabla actual y otra tabla en la base de datos.

Cuando se realiza una operación en una tabla que afecta a una clave primaria, como la inserción, actualización o borrado de un registro, la integridad referencial asegura que cualquier registro relacionado en otra tabla se actualice o elimine en consecuencia.

Si esta relación no se mantiene se producirían inconsistencias en la base de datos, lo que puede provocar errores y problemas en las aplicaciones que utilizan la base de datos.

Importante

Cuando se borran o modifican los registros en una tabla, es importante asegurarse de que las relaciones entre esa tabla y otras tablas relacionadas se mantengan intactas para mantener la integridad referencial de la base de datos.

MySQL ofrece varias opciones para manejar la integridad referencial en caso de borrado o modificación de datos:

- **ON DELETE CASCADE**: esta opción se utiliza en una relación de clave foránea y borra automáticamente los registros relacionados en otras tablas cuando se borra el registro correspondiente en la tabla principal.
- **ON DELETE SET NULL:** esta opción se utiliza en una relación de clave foránea y establece automáticamente los valores de las claves foráneas correspondientes en NULL cuando se borra el registro correspondiente en la tabla principal. **Para ello, es importante haber definido estas claves foráneas admitiendo valores NULL.**
- **ON UPDATE CASCADE:** esta opción se utiliza en una relación de clave foránea y actualiza automáticamente los valores de las claves foráneas correspondientes cuando se actualiza el valor de la clave primaria correspondiente en la tabla principal.
- **ON UPDATE SET NULL:** esta opción se utiliza en una relación de clave foránea y establece automáticamente los valores de las claves foráneas correspondientes en NULL cuando se actualiza el valor de la clave primaria correspondiente en la tabla principal. **Para ello, es importante haber definido estas claves foráneas admitiendo valores NULL**.

Partiendo de la definición de las siguientes tablas “Clientes” y “Facturas”, veamos cómo aplicaríamos cada una de estas cláusulas y el efecto que tendrían:

Punto de partida.

```
CREATE TABLE Clientes (
  id_cliente INT NOT NULL PRIMARY KEY,
  nombre_cliente VARCHAR(50),
  direccion_domicilio VARCHAR(100),
  dirección_cp VARCHAR(5)
);
CREATE TABLE Facturas (
  id_factura INT NOT NULL PRIMARY KEY,
  id_cliente INT,
  fecha_factura DATE,
  base DECIMAL(10,2),
 porcentaje_IVA DECIMAL(10,2),
importe_IVA DECIMAL(4,2),
total_factura_IVA DECIMAL(10,2),
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
);
```

Ejemplo 1: ON DELETE CASCADE.

Si aplicamos la cláusula **ON DELETE CASCADE** en la tabla Facturas, debemos modificar la definición de la clave foránea que hace referencia a la tabla Clientes para que esta tenga dicha cláusula.

La cláusula **ON DELETE CASCADE** en una clave foránea indica que cuando se elimine una fila de la tabla padre (Clientes en este caso), todas las filas relacionadas en la tabla hija (Facturas en este caso) también se eliminarán automáticamente.

Esto puede ser útil para asegurarse de que no queden registros huérfanos en la tabla hija después de eliminar un registro de la tabla padre.

La definición de la clave foránea en la tabla Facturas quedaría de la siguiente manera:

```
CREATE TABLE Facturas (
  id_factura      INT        NOT NULL PRIMARY KEY,
```

```
  id_cliente        INT,
  fecha_factura     DATE,
  base              DECIMAL(10,2),
  porcentaje_IVA    DECIMAL(10,2),
  importe_IVA       DECIMAL(10,2),
  total_factura_IVA DECIMAL(10,2),
  FOREIGN KEY (id_cliente)
    REFERENCES Clientes(id_cliente)
    ON DELETE CASCADE
);
```

Con esta modificación si se elimina un registro de la tabla Clientes, todas las facturas asociadas a ese cliente también se eliminarán automáticamente de la tabla Facturas.

Ejemplo 2: ON DELETE SET NULL.

Si aplicamos la cláusula **ON DELETE SET NULL** en la tabla Facturas debemos modificar la definición de la clave foránea que hace referencia a la tabla Clientes para que esta tenga dicha cláusula.

La cláusula **ON DELETE SET NULL** en una clave foránea indica que cuando se elimine una fila de la tabla padre (Clientes en este caso), todas las filas relacionadas en la tabla hija (Facturas en este caso) tendrán el valor de la clave foránea establecido en NULL automáticamente.

Esto puede ser útil para evitar errores al intentar eliminar un registro de la tabla padre que tenga registros relacionados en la tabla hija.

La definición de la clave foránea en la tabla Facturas quedaría de la siguiente manera:

```
CREATE TABLE Facturas (
  id_factura INT NOT NULL PRIMARY KEY,
  id_cliente INT,
  fecha_factura DATE,
  base DECIMAL(10,2),
```

```
  porcentaje_IVA DECIMAL(10,2),
  importe_IVA DECIMAL(4,2),
  total_factura_IVA DECIMAL(10,2),
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente) ON DELETE SET NULL
);
```

Con esta modificación, si se elimina un registro de la tabla Clientes, todas las facturas asociadas a ese cliente tendrán el valor de la clave foránea establecido en NULL automáticamente.

Hay que tener en cuenta que esta operación puede generar valores nulos en la tabla hija, por lo que se debe tener cuidado al utilizar la cláusula **ON DELETE SET NULL** y asegurarse de que la aplicación sea capaz de manejar valores nulos en la tabla hija.

Ejemplo 3: ON UPDATE CASCADE.

Para propagar cambios al actualizar Clientes.id_cliente, hay que modificar la **clave foránea** en la tabla **Facturas** que referencia a dicha clave, añadiendo ON UPDATE CASCADE.

La cláusula **ON UPDATE CASCADE** en una clave foránea indica que cuando se actualice el valor de una fila de la tabla padre (Clientes en este caso), todas las filas relacionadas en la tabla hija (Facturas en este caso) también se actualizarán automáticamente.

Esto puede ser útil para asegurarse de que los datos relacionados en la tabla hija reflejen los cambios realizados en la tabla padre.

La definición de la clave foránea en la tabla Facturas quedaría de la siguiente manera:

```
CREATE TABLE Facturas (
  id_factura INT NOT NULL PRIMARY KEY,
  id_cliente INT,
  fecha_factura DATE,
  base DECIMAL(10,2),
  porcentaje_IVA DECIMAL(10,2),
  importe_IVA DECIMAL(4,2),
```

```
 total_factura_IVA DECIMAL(10,2),
 FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente) ON UPDATE CASCADE
);
```

Con esta modificación, **si se actualiza el valor de la clave primaria en la tabla Clientes,** todas las facturas asociadas a ese cliente también se actualizarán automáticamente en la tabla Facturas.

Debemos tener en cuenta que esta operación puede afectar a una gran cantidad de datos, por lo que se debe tener cuidado al utilizar la cláusula **ON UPDATE CASCADE** y asegurarnos de que la aplicación pueda manejar correctamente los cambios en la tabla hija.

Ejemplo 4: ON UPDATE SET NULL.

Para que al actualizar Clientes.id_cliente los valores relacionados en Facturas pasen a NULL, hay que añadir ON UPDATE SET NULL a la **clave foránea** id_cliente de la tabla **Facturas**, no modificar la clave primaria de Clientes.

La cláusula **ON UPDATE SET NULL** en una clave foránea indica que cuando se actualice el valor de una fila de la tabla padre (Clientes en este caso), todas las filas relacionadas en la tabla hija (Facturas en este caso) tendrán el valor de la clave foránea actualizado a NULL automáticamente.

Esto puede ser útil para eliminar la relación entre dos tablas cuando se actualiza la clave primaria en la tabla padre.

La definición de la clave foránea en la tabla Facturas quedaría de la siguiente manera:

```
CREATE TABLE Facturas (
 id_factura INT NOT NULL PRIMARY KEY,
 id_cliente INT,
 fecha_factura DATE,
 base DECIMAL(10,2),
 porcentaje_IVA DECIMAL(10,2),
 importe_IVA DECIMAL(4,2),
 total_factura_IVA DECIMAL(10,2),
```

```
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente) ON UPDATE SET NULL
    );
```

4. Subconsultas y composiciones en órdenes de edición

4.1 Subconsultas de edición

Las subconsultas y composiciones se utilizan a menudo en órdenes de edición como UPDATE, INSERT o DELETE para obtener o modificar datos en tablas relacionadas. No olvidemos que las subconsultas son consultas SELECT anidadas dentro de otra consulta

Las subconsultas pueden ser utilizadas en una orden de edición para seleccionar un conjunto de registros que serán actualizados, eliminados o insertados en otra tabla.

Importante

Sintaxis básica.

Se puede utilizar una subconsulta en una orden **UPDATE** para modificar en una tabla en función de los valores de otra tabla.

```
UPDATE tabla1
SET columna1 = (SELECT columna2 FROM tabla2 WHERE tabla2.id = tabla1.id)
WHERE condicion;
```

En este ejemplo, la subconsulta se utiliza para obtener el valor de columna2 de la tabla2 en función del valor de id en la tabla1, y se actualiza el valor de columna1 en la tabla1 con ese valor.

Ejemplo: Subconsulta de edición e INSERT.

Supongamos que tenemos dos tablas: "ventas" y "productos".

La tabla "ventas" contiene información sobre las ventas realizadas, incluyendo el ID del producto vendido.

La tabla "productos" contiene información sobre los productos, incluyendo el precio de venta.

Queremos agregar una nueva venta a la tabla "ventas", pero en lugar de ingresar el precio de venta manualmente, queremos obtenerlo automáticamente de la tabla "productos" usando una subconsulta de edición.

El siguiente código SQL muestra cómo podemos hacer esto:

```
INSERT INTO ventas (id_producto, fecha, cantidad, precio)
VALUES (1, '2023-03-26', 10, (SELECT precio_venta FROM productos WHERE id = 1));
```

En este ejemplo, estamos agregando una nueva venta para el producto con ID 1. Estamos especificando la fecha de la venta y la cantidad vendida, y usando una subconsulta de edición para obtener el precio de venta del producto de la tabla "productos".

La subconsulta se encuentra entre paréntesis dentro de la cláusula VALUES.

La subconsulta devuelve el precio de venta del producto con ID 1, que luego se inserta en la columna "precio" de la tabla "ventas".

Ejemplo: Subconsulta de edición y UPDATE.

Supongamos que tenemos las mismas tablas "ventas" y "productos" que en el ejemplo anterior.

Esta vez, queremos actualizar la tabla "ventas" para cambiar el precio de venta de un producto vendido anteriormente, y queremos obtener el nuevo precio automáticamente de la tabla "productos" utilizando una subconsulta de edición.

El siguiente código SQL muestra cómo podemos hacer esto:

```
UPDATE ventas
SET precio = (SELECT precio_venta FROM productos WHERE id = 1)
WHERE id_producto = 1 AND fecha = '2023-03-26';
```

En este ejemplo, estamos actualizando la tabla "ventas" para cambiar el precio de venta de un producto vendido en la fecha '2023-03-26'. Estamos utilizando una subconsulta de edición para obtener el nuevo precio de venta del producto con ID 1 de la tabla "productos".

La subconsulta se encuentra entre paréntesis después de la palabra SET.

La subconsulta devuelve el nuevo precio de venta del producto con ID 1, que luego se utiliza para actualizar la columna "precio" de la tabla "ventas" en las filas que cumplen con la condición especificada en la cláusula WHERE.

La condición WHERE en este ejemplo asegura que sólo se actualizarán los registros que correspondan al producto con ID 1 y la fecha especificada.

Ejemplo: Subconsulta de edición y DELETE.

Supongamos que tenemos las mismas tablas "ventas" y "productos" que en los ejemplos anteriores.

Esta vez, queremos eliminar todas las ventas registradas en la tabla "ventas" para un producto específico, cuyo precio de venta es menor a un cierto valor, y queremos obtener el ID del producto automáticamente de la tabla "productos" utilizando una subconsulta de edición.

El siguiente código SQL muestra cómo podemos hacer esto:

```
DELETE FROM ventas
WHERE id_producto = (SELECT id FROM productos WHERE precio_venta < 100);
```

En este ejemplo, estamos utilizando una subconsulta de edición dentro de la cláusula WHERE para obtener el ID del producto cuyo precio de venta es menor a 100.

La subconsulta se encuentra entre paréntesis después del signo igual en la condición WHERE.

Estamos eliminando todos los registros de la tabla "ventas" que correspondan a ese producto utilizando la cláusula DELETE.

4.2 Composiciones

Las composiciones (o Common Table Expressions, CTE) permiten encadenar varias subconsultas nombradas al inicio de una sentencia SQL y luego reutilizarlas en una sola operación de edición (UPDATE, DELETE o INSERT). Mejoran la legibilidad y evitan repetir subconsultas complejas.

¿Qué es un CTE?:

Se declara con `WITH nombre_cte AS ( … )` antes de la consulta principal.

Cada CTE actúa como una “tabla temporal” que contiene el resultado de la subconsulta.

MySQL 8.0+ soporta CTEs en `SELECT`, `UPDATE`, `DELETE` e `INSERT`.

Sintaxis básica de CTEs:

```
WITH
 cte1 AS (
  SELECT columnas
  FROM tabla_origen1
  WHERE condiciones1
 ),
 cte2 AS (
  SELECT columnas
  FROM tabla_origen2
  WHERE condiciones2
 )
```

Aquí va la consulta principal (SELECT, UPDATE, DELETE o INSERT):

```
SELECT *
FROM cte1
JOIN cte2 USING (columna_común);
```

Supongamos tres tablas con la misma clave id:

tabla1(id, columna1)

tabla2(id, columna4)

tabla3(id, columna5, columna6)

Queremos cargar en dos CTEs los valores de columna1 y columna4, y luego actualizar tabla3 asignando esos datos:

```
WITH
 c1 AS (
  SELECT id, columna1
  FROM tabla1
  WHERE condicion1
 ),
 c2 AS (
```

```
  SELECT id, columna4
  FROM tabla2
  WHERE condicion2
 )
UPDATE tabla3 t
 JOIN c1 ON t.id = c1.id
 JOIN c2 ON t.id = c2.id
SET
 t.columna5 = c1.columna1,
 t.columna6 = c2.columna4;
```

Cómo funciona:

La CTE c1 extrae (id, columna1) de tabla1.

La CTE c2 extrae (id, columna4) de tabla2.

El UPDATE une tabla3 con c1 y c2 usando id y asigna:

tabla3.columna5 ← c1.columna1

tabla3.columna6 ← c2.columna4

De esta manera, combinamos varias subconsultas y las aplicamos en un único bloque de edición, manteniendo el código claro y eficiente.

5. Transacciones. Sentencias de confirmación y deshacer

Antes de trabajar con transacciones que dependan de errores de datos (por ejemplo, convertir cadenas no numéricas a números), es recomendable activar el modo estricto para la sesión actual:

```
SET SESSION sql_mode = 'STRICT_ALL_TABLES';
```

sql_mode es una variable de MySQL que controla validaciones y conversiones de datos.

En modo **STRICT_ALL_TABLES**, MySQL rechazará operaciones que impliquen conversiones inválidas (como sumar 'abc' a un DECIMAL) o truncan valores, en lugar de aplicarlas silenciosamente.

Las transacciones son un conjunto de operaciones SQL que se ejecutan como una sola unidad lógica y que, en conjunto, representan una tarea completa.

Las transacciones se utilizan para asegurar la integridad de los datos y para garantizar que todas las operaciones se completen o se anulen en su totalidad.

En MySQL, las transacciones se pueden controlar mediante las sentencias COMMIT y ROLLBACK, que son utilizadas para confirmar o deshacer una transacción, respectivamente:

- -**COMMIT:** La sentencia COMMIT se utiliza para confirmar una transacción. Cuando se ejecuta la sentencia COMMIT, todas las operaciones que se realizaron dentro de la transacción se guardan de manera permanente en la base de datos.
- -**ROLLBACK:** La sentencia ROLLBACK se utiliza para deshacer una transacción. Cuando se ejecuta la sentencia ROLLBACK, todas las operaciones que se realizaron dentro de la transacción se deshacen, es decir, se revierten los cambios realizados y se devuelve la base de datos al estado que tenía antes de que comenzara la transacción.

Ejemplo.

Supongamos que tenemos una tabla "cuentas" que contiene información sobre las cuentas bancarias de nuestros clientes.

Queremos actualizar el saldo de dos cuentas haciendo una transferencia: una que pertenece a Juan Pérez y otra que pertenece a María Gómez.

Si la actualización se completa correctamente, confirmamos los cambios utilizando la sentencia COMMIT.

Si algo sale mal, podemos deshacer los cambios utilizando la sentencia ROLLBACK.

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 500 WHERE cliente = 'Juan Pérez';
UPDATE cuentas SET saldo = saldo + 500 WHERE cliente = 'María Gómez';
COMMIT;
```

En este ejemplo estamos utilizando la sentencia START TRANSACTION para iniciar la transacción. Luego, estamos actualizando el saldo de dos cuentas diferentes en la tabla "cuentas".

En la primera actualización, estamos restando 500 unidades del saldo de la cuenta de Juan Pérez. En la segunda actualización, estamos sumando 500 unidades al saldo de la cuenta de María Gómez.

Después de las dos actualizaciones, confirmamos los cambios utilizando la sentencia **COMMIT**.

Si algo saliera mal durante la transacción, podríamos utilizar la sentencia **ROLLBACK** para deshacer las operaciones realizadas hasta ese momento:

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 500 WHERE cliente = 'Juan Pérez';
UPDATE cuentas SET saldo = saldo + 'abc' WHERE cliente = 'María Gómez';
ROLLBACK;
```

En este ejemplo, estamos realizando las mismas operaciones que en el ejemplo anterior, pero esta vez estamos intentando sumar un valor no numérico en la columna "saldo" de la cuenta de María Gómez.

Si ejecutamos esta transacción, la actualización fallará y la sentencia ROLLBACK deshará la primera actualización realizada y dejará la base de datos en el estado que tenía antes de comenzar la transacción.

Tech Hub <FP> euroformac

6. Políticas de bloqueo: a nivel de registro, a nivel de tabla

Las políticas de bloqueo son mecanismos utilizados por los sistemas de bases de datos para controlar el acceso concurrente a los datos y prevenir conflictos y errores de consistencia.

Existen dos niveles principales de políticas de bloqueo: a nivel de registro y a nivel de tabla.

- **A nivel de registro**, también conocido como **bloqueo de fila**, se bloquea el acceso a un registro específico en una tabla.

  Esto significa que otros usuarios no pueden leer o modificar ese registro mientras el bloqueo esté activo. El bloqueo se mantiene hasta que se completa la operación en el registro bloqueado o hasta que el usuario libera el bloqueo manualmente.

  Se suele usar en situaciones en las que se están actualizando registros específicos en una tabla y se quiere asegurar que no se realicen cambios concurrentes en el mismo registro. Pero también puede llevar a bloqueos prolongados y conflictos si muchos usuarios están intentando acceder al mismo registro al mismo tiempo.

- A **nivel de tabla**, se bloquea el acceso a toda la tabla en lugar de a registros individuales.

  Esto significa que otros usuarios no pueden leer o modificar ningún registro en la tabla mientras el bloqueo esté activo. El bloqueo se mantiene hasta que se completa la operación en la tabla bloqueada o hasta que el usuario libera el bloqueo manualmente.

  Se usa cuando las operaciones que afectan a toda la tabla, como la creación o eliminación de tablas, o la realización de copias de seguridad.

  Es muy inconveniente y puede causar interrupciones significativas en la base de datos si muchos usuarios intentan acceder a la tabla bloqueada al mismo tiempo.

Ejemplo.

Supongamos que tenemos una tabla llamada "ventas" que almacena información sobre las ventas realizadas en una tienda.

Esta tabla tiene una columna llamada "id_venta" que es una clave primaria auto incremental y otra columna llamada "importante" que almacena el valor total de la venta.

Supongamos ahora, que dos usuarios diferentes están intentando actualizar la misma venta al mismo tiempo.

Para evitar que ocurran conflictos de datos, podemos utilizar el bloqueo de registro para garantizar que solo un usuario tenga acceso al registro en un momento dado.

Para bloquear un registro en MySQL, podemos utilizar la siguiente sintaxis:

```
START TRANSACTION;

SELECT * FROM ventas WHERE id_venta = 100 FOR UPDATE;

-- Realizar la operación de actualización en el registro bloqueado aquí

UPDATE ventas SET importe = importe + 50 WHERE id_venta = 100;

COMMIT;
```

Los pasos que hemos seguido son:

- Hemos iniciado una nueva transacción con la sentencia START TRANSACTION.
- Seleccionamos el registro de la venta con "id_venta" igual a 100 utilizando la cláusula **FOR UPDATE** para bloquearlo y evitar que otros usuarios puedan modificarlo mientras realizamos la actualización.
- Después de bloquear el registro, realizamos la operación de actualización en el registro bloqueado. En este caso, estamos incrementando el valor de la columna "importe" en 50.
- Finalmente, confirmamos los cambios utilizando la sentencia COMMIT para completar la transacción.

Importante

La actualización solo afectará al registro bloqueado por la transacción actual, y cualquier otro usuario que intente acceder al mismo registro mientras está bloqueado tendrá que esperar a que se complete la transacción antes de poder realizar cambios.

Siguiendo con el ejemplo práctico veamos cómo se resuelve la gestión de miembros, membresías y reservas en FitLife

EL gimnasio FitLife mantiene tres tablas básicas en MySQL:

```
CREATE TABLE Miembros (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(50),
  apellidos   VARCHAR(100),
  fecha_alta  DATE
);

CREATE TABLE Membresías (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  miembro_id  INT,
  tipo        VARCHAR(20),     -- 'Mensual', 'Trimestral', 'Anual'
  fecha_inicio  DATE,
  fecha_fin   DATE,
  FOREIGN KEY (miembro_id) REFERENCES Miembros(id)
);

CREATE TABLE ReservasClases (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  miembro_id  INT,
```

```
  fecha_clase   DATE,
  nombre_clase  VARCHAR(50),
  FOREIGN KEY (miembro_id) REFERENCES Miembros(id)
);
```

En MySQL es necesario realizar las siguientes tareas:

1. Dar de alta a "Ana López" (fecha de alta hoy) y asignarle una membresía Anual que empiece hoy y termine dentro de un año.
2. "Ana López" desea cambiar su membresía de "Anual" a "Trimestral". Actualiza su registro de membresía ajustando tipo y fecha_fin.
3. Eliminar todas las reservas de clase cuya fecha_clase sea anterior a 30 días atrás (es decir, estrictamente anterior a la fecha actual menos 30 días).
4. Supongamos que el gimnasio quiere transferir a "Ana López" dos días extra de clase de su membresía trimestral a la membresía de "Carlos Ruiz". Debe ser una operación atómica: si falla alguna parte, se deshace todo.
5. Dos instructores podrían intentar asignar plaza en la misma clase del 2025-07-20 a un nuevo socio "Luis Martín". Muestra cómo bloquear la fila de ReservasClases para esa fecha antes de insertar o verificar disponibilidad.

Soluciones

1. **Alta de nuevo miembro y asignación de membresía (INSERT)**

1.1 Insertar nuevo miembro

```
INSERT INTO Miembros (nombre, apellidos, fecha_alta)
VALUES ('Ana', 'López', CURDATE());
```

1.2 Asignar membresía Anual

```
INSERT INTO Membresías (miembro_id, tipo, fecha_inicio, fecha_fin)
VALUES (
  LAST_INSERT_ID(),
  'Anual',
  CURDATE(),
  DATE_ADD(CURDATE(), INTERVAL 1 YEAR)
);
```

2. **Actualización de membresía (UPDATE)**

```
UPDATE Membresías m
JOIN Miembros p ON m.miembro_id = p.id
SET
  m.tipo     = 'Trimestral',
  m.fecha_fin = DATE_ADD(m.fecha_inicio, INTERVAL 3 MONTH)
WHERE
  p.nombre = 'Ana'
  AND p.apellidos = 'López';
```

3. **Limpieza de reservas antiguas (DELETE)**

```
DELETE FROM ReservasClases
WHERE fecha_clase < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

4. **Transferencia atómica de días de clase (TRANSACCIÓN + ROLLBACK)**

```
SET SESSION sql_mode = 'STRICT_ALL_TABLES';  -- para asegurar errores si algo va mal
START TRANSACTION;
-- Restar 2 días de la membresía de Ana López
UPDATE Membresías m
JOIN Miembros p ON m.miembro_id = p.id
SET m.fecha_fin = DATE_SUB(m.fecha_fin, INTERVAL 2 DAY)
WHERE p.nombre = 'Ana' AND p.apellidos = 'López';

-- Añadir 2 días a la membresía de Carlos Ruiz
UPDATE Membresías m
JOIN Miembros p ON m.miembro_id = p.id
SET m.fecha_fin = DATE_ADD(m.fecha_fin, INTERVAL 2 DAY)
WHERE p.nombre = 'Carlos' AND p.apellidos = 'Ruiz';
```

```
-- Si ambas UPDATE tienen éxito:
COMMIT;
/* Si alguna falla (por ejemplo, modo estricto detecta error),
   la aplicación o el script debe capturar la excepción y emitir:
   ROLLBACK;
*/
```

5. Evitar doble reserva simultánea (BLOQUEO DE FILA)

```
START TRANSACTION;

5.1 Bloquear cualquier fila para la clase del 2025-07-20
SELECT *
FROM ReservasClases
WHERE fecha_clase = '2025-07-20'
FOR UPDATE;

5.2 Verificar disponibilidad (por ejemplo, máximo 20 plazas)
SELECT COUNT(*) INTO @plazas_ocupadas
FROM ReservasClases
WHERE fecha_clase = '2025-07-20';

5.3 Si queda plaza, insertar nueva reserva
IF @plazas_ocupadas < 20 THEN
  INSERT INTO ReservasClases (miembro_id, fecha_clase, nombre_clase)
  VALUES (
    (SELECT id FROM Miembros WHERE nombre='Luis' AND apellidos='Martín'),
    '2025-07-20',
    'Yoga Avanzado'
  );
END IF;
```

COMMIT;

Con este caso práctico se aplican:

- **Inserción, actualización y borrado** de registros.
- **Control transaccional** con START TRANSACTION / COMMIT / ROLLBACK.
- **Bloqueo de fila** (SELECT … FOR UPDATE) para gestionar concurrencia.

7. Resumen

La **inserción, borrado y modificación de registros en MySQL** se puede realizar utilizando sentencias SQL como INSERT, DELETE y UPDATE. Estas operaciones se pueden realizar a partir de datos proporcionados por el usuario o recuperados mediante subconsultas.

Los **borrados y modificaciones en MySQL** pueden afectar a la integridad referencial de la base de datos si no se toman las precauciones adecuadas. La integridad referencial se puede mantener utilizando restricciones como claves foráneas y cascadas de borrado y actualización.

Las **subconsultas y composiciones en órdenes de edición en MySQL** permiten realizar operaciones más complejas en la base de datos, como la actualización de registros utilizando valores de otras tablas o la inserción de registros basados en el resultado de una consulta.

Las **transacciones en MySQL** permiten agrupar un conjunto de operaciones en una sola unidad lógica para garantizar la consistencia de la base de datos. Las sentencias COMMIT y ROLLBACK se utilizan para confirmar o deshacer los cambios realizados en una transacción.

Las **políticas de bloqueo en MySQL** permiten controlar el acceso a los registros y tablas de la base de datos para evitar conflictos de datos entre usuarios concurrentes. El bloqueo a nivel de registro y a nivel de tabla se puede utilizar para garantizar la integridad de los datos en situaciones de alta concurrencia.

Glosario

- **Inserción**: acción de agregar un nuevo registro a una tabla de una base de datos.
- **Borrado**: acción de eliminar un registro de una tabla de una base de datos
- **Modificación**: acción de actualizar o cambiar los valores de un registro existente en una tabla de una base de datos.
- **Integridad referencial**: restricción que asegura que los valores en una tabla se corresponden con los valores en otra tabla relacionada
- **Subconsulta**: consulta anidada dentro de otra consulta.
- **Composiciones en órdenes de edición**: combinación de varias operaciones de edición (como inserción, borrado y modificación) en una única instrucción de edición.
- **Transacciones**: conjunto de operaciones de base de datos que se ejecutan como una única unidad lógica.
- **Sentencias de confirmación y deshacer**: sentencias utilizadas para confirmar o deshacer las transacciones de base de datos.
- **Políticas de bloqueo**: estrategias utilizadas para controlar el acceso concurrente a los datos en una base de datos. Las políticas de bloqueo pueden ser a nivel de registro o a nivel de tabla, y se utilizan para prevenir conflictos de acceso y garantizar la integridad de los datos.

Bibliografía

Kroneke, D. M. (1996). Procesamiento de bases de datos. Fundamentos, diseño e instrumentación (5ª ed.). Prentice-Hall.

Silberschatz, A., Korth, H., & Sudarshan, S. (1998). Fundamentos de bases de datos (3ª ed.). McGraw-Hill.

Codd, E. F. (1986). Missing information (applicable and inapplicable) in relational databases. SIGMOD Record, 15(4).

Tech Hub <FP> euroformac